



WEBSITE REQUIREMENT DOCUMENT (WRD)

Full Stack Developing — Frontend · Backend · Admin Panel
| April 2026 | CONFIDENTIAL

Document Field	Details
Project Name	Vibe Music — Music Gear eCommerce Platform
Document Type	Website Requirement Document (WRD)
Version	
Date	April 2026
Prepared By	Samiran Kha (Senior Ecom Product Manager & Full-Stack Architect)
Intended Audience	Frontend, Backend & Admin Panel Developers
Confidentiality	Internal Development Use Only

SECTION 01 PROJECT OVERVIEW

1.1 Purpose & Vision

Vibe Music is a full-featured, B2C eCommerce platform for musical instruments, recording equipment, audio gear, and accessories — inspired by Sweetwater.com. The platform combines a high-performance online store Providing customers with both self-service shopping and expert human guidance.

The goal is to build a platform that musicians trust: one that matches technical depth with ease of use, and that treats every customer as a valued artist rather than an anonymous buyer.

1.2 Target Users

- Amateur and professional musicians (guitarists, drummers, pianists, producers)
- Home studio owners and bedroom producers
- Professional recording studios and sound engineers
- Schools, colleges, and music institutions (B2B purchasers)
- Live sound professionals (FOH engineers, touring musicians)
- Content creators and podcasters requiring audio gear

1.3 Business Model

- Direct product sales (instruments, gear, accessories, software)
- Bundled product packages (e.g., guitar + amp + cable starter pack)
- Manufacturer partnerships and exclusive product deals
- Extended warranty and gear protection plans (upsell)
- Personal Sales Engineer consultations (differentiator)
- Educational content and gear guides (SEO + retention)

1.4 Technical Ambition

Dimension	Goal	Priority
Performance	< 2s page load (LCP), Core Web Vitals green	Critical
Scale	Support 100k+ products, 50k+ concurrent users	High
SEO	Server-side rendered product & category pages	Critical
Security	PCI-DSS compliant checkout, JWT auth, input sanitisation	Critical
Accessibility	WCAG 2.1 AA compliant	High
Mobile	Mobile-first responsive, PWA-ready	High

SECTION 02 COMPLETE FEATURE LIST

2.1 Customer-Facing Features

Homepage & Navigation

- Sticky top navigation bar with logo, search, cart icon, account menu
- Mega menu: multi-column dropdown with categories, subcategories, featured brands, and promotional banners
- Full-width hero banner carousel with CTA buttons (auto-play, pause on hover)
- Curated product sections: New Arrivals, Best Sellers, Staff Picks, Deal of the Day
- Brand logo strip (horizontal scroll on mobile)
- Featured categories grid (visual tiles with icon + label)
- Sales Engineer CTA banner (personalised support teaser)
- Footer: links, newsletter signup, social media, payment badges, trust badges

Search System

- Real-time autocomplete search (debounced, 300ms, minimum 2 chars)
- Search suggestions: products, categories, brands, and recent searches
- Full search results page with faceted filters and sorting
- Zero-results fallback with recommendations and search tips
- Search analytics logged for admin review

Category & Browse

- SEO-friendly category URLs (e.g., /guitars/electric-guitars)
- Breadcrumb navigation on all category and product pages
- Left sidebar: multi-select filters (brand, price range, condition, availability, rating)
- Sort options: Featured, Price Low-High, Price High-Low, Best Sellers, Newest, Rating
- Product grid (3-col desktop / 2-col tablet / 1-col mobile) with lazy load
- Grid / List view toggle
- Active filter chips with individual remove and 'Clear All'
- Product count display ('Showing 1–24 of 1,842 results')
- Pagination with page numbers + infinite scroll option

Product Page

- Image gallery: main image, thumbnails, zoom on hover, full-screen lightbox
- 360-degree image support and embedded product video (YouTube/Vimeo)
- Product title, SKU, brand, condition badge
- Price with sale price, discount percentage, and savings amount
- Availability indicator: In Stock / Low Stock / Out of Stock / Backorder ETA
- Variant selector: colour, finish, size (linked to individual SKUs)
- Quantity selector with stock limit enforcement
- Add to Cart button (animated) and Add to Wishlist
- Buy Now
- Shipping estimate: zip/pin code entry → live estimated delivery date
- Tabbed content: Description, Specs, In the Box, Reviews, Q&A, Related Videos

- Full product specification table (manufacturer specs)
- Customer reviews section: star ratings, verified purchase badge, helpful votes
- Questions & Answers section with user submissions
- Frequently Bought Together (cross-sell bundle)
- Similar Products carousel
- Sales Engineer Card: assigned agent photo, name, direct contact options

Cart & Wishlist

- Persistent cart (stored in backend, synced across devices on login)
- Mini cart drawer (slide-in from right, available on all pages)
- Cart page: item list, quantity edit, remove, price breakdown
- Coupon/promo code field with real-time validation
- Order summary: subtotal, shipping, tax, total
- Wishlist: save for later, share wishlist link, move to cart
- Recently Viewed products persistent section

Checkout System

- Guest checkout option (no forced registration)
- Step 1 – Contact & Shipping: name, email, phone, address with autocomplete (Google Places API)
- Step 2 – Shipping method selection with pricing (Standard, Express, Overnight)
- Step 3 – Payment: Stripe card, Razorpay UPI/Net Banking
- Step 4 – Order Review and Place Order
- Step 5 – Confirmation: order number, summary email, estimated delivery
- Order tracking page with real-time status timeline

User Account Dashboard

- Profile management (name, photo, password, communication preferences)
- Order history with status, invoice download, and return request initiation
- Saved addresses (multiple, default flag)
- Payment methods (tokenised card management)
- Wishlist management
- Loyalty points / reward balance (if loyalty module enabled)
- Notification preferences (email, SMS, push)

2.2 Admin Panel Features

- Dashboard: KPIs, revenue charts, live order feed, low-stock alerts
- Product management: add, edit, delete, bulk import/export via CSV
- Category & attribute management: hierarchical categories, custom attributes
- Order management: view, filter, update status, refund, partial fulfil
- Customer management: profiles, order history, flag/ban,
- Review moderation: approve, reject, reply, flag for abuse
- Blog & content editor: rich-text CMS with SEO fields
- Banner and homepage content manager (drag-and-drop)
- Coupon and promotions management
- Shipping zone and rate configuration
- Analytics: sales reports, traffic, search terms, conversion funnel
- Inventory & stock management with low-stock email alerts

-
- Role & permissions management (Admin, Manager, Support)

2.3 Sales Engineer System (Key Differentiator) (**NOT REQUIRED**)

The Sales Engineer (SE) system is Vibe Music most important differentiator. It mirrors Vibe Music famous personal support model, where each customer is assigned a dedicated knowledgeable sales person.

- Every registered customer is auto-assigned a Sales Engineer based on workload balancing
- SE profile card visible on product pages, cart, and account dashboard
- Customer can contact SE via: in-app message, email, phone callback request
- SE has a dedicated CRM dashboard: their customer list, recent orders, open conversations
- Admin can re-assign SE, view SE performance (revenue, response time, CSAT)
- SE can leave internal notes per customer visible only to admin/SE team
- SE can recommend products and create personalised bundles for their customer

SECTION 03 PAGE-BY-PAGE BREAKDOWN

3.1 Homepage (/)

Purpose	The primary landing page and marketing hub. Drive discovery, inspire purchase intent, and showcase curated content.
---------	---

UI Sections

- Sticky Header: Logo (left), Search bar (center), Icons (right: wishlist, account, cart with badge)
- Hero Banner Carousel: Full-width, 3–5 slides, image + headline + CTA, autoplay 5s, dots nav
- Featured Categories: 6–8 visual category tiles in a 4-col grid
- New Arrivals Strip: Horizontal scrollable product cards with 'View All' CTA
- Best Sellers Grid: 8-product 4-col grid
- Brand Logos Strip: Horizontal scrollable, 12+ brand logos
- Deal of the Day: Single product highlight with countdown timer
- Blog/Gear Guides: 3-col card grid with article preview
- Footer: Sitemap links, contact info, newsletter, social icons, trust badges

User Actions

- Search, click category, click product, add to cart from quick-view, navigate to SE contact

Backend Data Required

- GET /banners (active homepage banners)
- GET /categories?featured=true
- GET /products?type=new_arrivals&limit=12
- GET /products?type=best_sellers&limit=8
- GET /deals/today
- GET /brands?featured=true
- GET /blog/posts?limit=3&category=featured

3.2 Category Page (/category/:slug)

Purpose

Browse and filter all products within a category or subcategory. Primary conversion page.

UI Sections

- Breadcrumb trail: Home > Guitars > Electric Guitars
- Page title + product count
- Left sidebar (desktop) / Drawer (mobile): Filter groups (Brand, Price Slider, Rating, Availability, Condition)
- Active filter chips bar with 'Clear All' button
- Sort dropdown (top right): Featured, Price, Newest, Rating
- Grid/List toggle
- Product cards: image, name, brand, price, rating stars, quick-add-to-cart
- Pagination bar / 'Load More' button
- Sub-category pill navigation (horizontal scroll)

User Actions

- Apply filters (multi-select), clear filters, sort, toggle view, click product, quick-add to cart, paginate

Backend Data Required

- GET /categories/:slug (category info, subcategories)
- GET /products?category=:slug&brand=:b&min_price=:min&max_price=:max&rating=:r&sort=:s&page=:p&limit=24
- GET /filters/category/:slug (available filter options and counts)

3.3 Product Page (/product/:slug)

Purpose

Provide all information a customer needs to make a confident purchase decision.

UI Sections

- Breadcrumb
- Left: Image gallery (main + thumbnails, zoom, lightbox, 360°/video toggle)
- Right: Brand, title, SKU, ratings summary, price block, variant selector, quantity, Add to Cart, Buy Now, Add to Wishlist, Instalment estimate, Shipping estimate widget, Sales Engineer card
- Tabs below: Description | Specs | In the Box | Reviews | Q&A | Videos
- Frequently Bought Together: 2–3 product suggestion with combined add-to-cart
- Similar Products carousel

User Actions

- Select variant, enter quantity, add to cart, add to wishlist, enter pin for shipping, submit review, ask question, contact SE

Backend Data Required

- GET /products/:slug (full product data, images, specs, variants)
- GET /products/:id/reviews?page=:p&sort=:s
- GET /products/:id/related
- GET /products/:id/frequently-bought-with
- GET /shipping/estimate?pin=:pin&productId=:id
- GET /users/:userId/sales-engineer (SE card info)
-

3.4 Cart Page (/cart)

Purpose

Review selected items, adjust quantities, apply coupons, and proceed to checkout.

UI Sections

- Cart item rows: thumbnail, name, variant, unit price, quantity stepper, line total, remove button
- Coupon code field with 'Apply' button and validation messages
- Order Summary panel: subtotal, discount, estimated shipping, tax, total
- Proceed to Checkout CTA (primary, full-width)
- Continue Shopping link
- Recommended Products section (personalised)
- Recently Viewed strip

Backend Data Required

- GET /cart (user's active cart with items)
- PUT /cart/items/:itemId (update quantity)
- DELETE /cart/items/:itemId
- POST /cart/coupon (apply coupon)
- GET /products/recommendations?context=cart&productIds=:ids

3.5 Checkout Page (/checkout)

Purpose

Guide the user through address, shipping, and payment steps to complete purchase.

Step-by-Step UI Breakdown

1. Step 1 – Contact & Address: email, full name, phone, address line 1/2, city, state, zip, country. Google Places autocomplete.
2. Step 2 – Shipping Method: radio buttons for Standard Free, Express (price), Overnight (price) with estimated dates
3. Step 3 – Payment: Stripe card form (embedded), Razorpay (redirect), Show order summary sidebar.
4. Step 4 – Review: full summary. Place Order button. Show terms link.
5. Step 5 – Confirmation: animated success, order number, items list, email confirmation notice, 'Track Your Order' CTA.

Backend Data Required

- GET /cart (latest cart state)
- GET /addresses (user saved addresses)
- POST /addresses (save new address)
- GET /shipping/rates?cartId=:id&addressId=:id
- POST /orders/create (create order, validate stock, lock cart)
- POST /payments/initiate (get payment intent/token)
- POST /payments/confirm (webhook-verified)

3.6 User Dashboard (/account)

Purpose

Centralised account management hub for the customer.

Sub-Pages / Tabs

- Overview: welcome message, recent orders, [SE card](#), quick links
- Orders: paginated order list with status badges, track link, return/exchange CTA
- Order Detail (/account/orders/:id): items, pricing, [shipment tracking timeline](#)
- Profile: edit name, email, phone, avatar upload
- Addresses: add/edit/delete shipping addresses
- Payment Methods: view tokenised saved cards, add/remove
- Wishlist: product grid from wishlist, move to cart, remove
- My SE: SE profile, message history, request callback
- Notifications: toggle preferences for email/SMS/push per event type

3.7 Admin Dashboard (/admin)

Purpose
Central operational hub for store management, order processing, and analytics.

UI Sections

- Top bar: notifications bell, admin profile dropdown, dark/light toggle
- Left sidebar: navigation to all modules
- Dashboard home: Revenue (today/week/month), Orders (new/pending/shipped), Customers (new today), Top Products, Low Stock alerts, Recent Order feed, Revenue chart (line/bar)
- Each module (Products, Orders, Customers, Reviews, Blog, SE, Settings) in dedicated sub-pages

SECTION 04 FRONTEND REQUIREMENTS

4.1 Technology Stack (I MENTIONED THE CHOICES IF YOU COMFORTABLE WITH ANY OTHER YOU CAN DO IT BY YOU OWN WHICH IS COMPACT AND BEST)

Technology	Choice & Justification
Framework	Next.js 14+ (App Router) — SSR for SEO on product/category pages, CSR for dashboard
Language	TypeScript — type safety, better DX, auto-docs
Styling	Tailwind CSS + shadcn/ui component library
State Management	Zustand (client state: cart, UI) + React Query / TanStack Query (server state, caching)
Forms	React Hook Form + Zod schema validation
Image Handling	Next/Image component with lazy loading and responsive srcsets
Icons	Lucide React
Payment UI	Stripe Elements (embedded card form)
Maps/Autocomplete	Google Places API (address autocomplete)
Testing	Vitest + React Testing Library + Playwright (E2E)
Linting	ESLint + Prettier + Husky pre-commit hooks

4.2 Project & Component Structure

```
src/
  app/                                # Next.js App Router pages
    (store)/                          # Public storefront layout group
      page.tsx                        # Homepage
      category/[slug]/page.tsx       # Category listing
      product/[slug]/page.tsx        # Product detail
      cart/page.tsx
      checkout/page.tsx
      account/
        orders/page.tsx
        profile/page.tsx
        wishlist/page.tsx
      se/page.tsx                    # My Sales Engineer
    (admin)/                          # Admin panel layout group
```

```

  dashboard/page.tsx
  products/page.tsx
  orders/page.tsx
  customers/page.tsx
  reviews/page.tsx
  blog/page.tsx
  se-panel/page.tsx
api/                                # Next.js API Routes (optional BFF layer)
components/
  layout/                          # Header, Footer, MegaMenu, Sidebar
  ui/                              # shadcn/ui primitives (Button, Input, Modal...)
  product/                        # ProductCard, ProductGallery, ReviewCard
  cart/                           # CartDrawer, CartItem, OrderSummary
  checkout/                       # CheckoutStepper, PaymentForm, AddressForm
  admin/                          # AdminSidebar, DataTable, StatsCard, ChartWidget
  se/                             # SECard, SEMessageThread
hooks/                            # useCart, useAuth, useWishlist, useSearch
lib/                              # apiClient.ts, constants.ts, utils.ts
store/                            # Zustand slices: cartStore, authStore, uiStore
types/                            # Global TypeScript interfaces
styles/                           # globals.css, tailwind config
```

4.3 Responsive Design Breakpoints

Breakpoint	Target & Behaviour
xs: < 480px	Single column layout, bottom navigation bar, full-width cards
sm: 480–767px	2-column product grid, hamburger menu, drawer filters
md: 768–1023px	2-col product grid, top nav visible, collapsed sidebar
lg: 1024–1279px	3-col product grid, full navigation, visible sidebar
xl: 1280px+	4-col product grid, mega menu, full desktop layout

4.4 UI Behaviour Specifications

Interactive States — Every interactive element must implement:

- Default → Hover (colour shift + cursor change)
- Focus (visible ring for accessibility — Tailwind ring-2 ring-blue-500)
- Active / Pressed (scale-95 transform)
- Disabled (opacity-50, cursor-not-allowed, pointer-events-none)
- Loading (skeleton placeholders or spinner overlay)
- Error (red border + helper text below field)
- Success (green confirmation toast, 3s auto-dismiss)

Loading States

- Product grid: Skeleton cards matching grid layout (Tailwind animate-pulse)
- Product page: Skeleton for gallery + details above fold while below-fold loads
- Search: Spinner inside search input, suggestion list appears < 300ms
- Checkout: Full overlay loader on payment submission (prevent double-submit)
- Admin tables: Skeleton rows while data fetches

Toast Notifications

- 'Added to cart' — product thumbnail + name, 'View Cart' action — 3s
- 'Removed from wishlist' — undo action — 5s
- 'Order placed successfully!' — success — 5s
- Error toasts — red, persist until dismissed

4.5 Form Validations

Form	Field	Validation Rule
Signup	Email	Required, valid email format, must be unique (async check)
Signup	Password	Min 8 chars, 1 uppercase, 1 number, 1 special char
Signup	Confirm Password	Must match password field
Checkout	Phone	Valid 10-digit number (India) or international E.164 format
Checkout	ZIP/PIN	5-digit US or 6-digit India, validated via shipping API
Checkout	Card Number	Stripe Elements handles — do not implement manually
Product Review	Rating	Required (1–5 stars)
Product Review	Body	Min 20 chars, max 1000 chars
Admin — Product	Price	Positive number, max 2 decimal places
Admin — Product	SKU	Unique, alphanumeric, 4–20 chars

4.6 API Integration Pattern

```
// lib/apiClient.ts - Centralised Axios instance
import axios from 'axios';

const api = axios.create({
  baseURL: process.env.NEXT_PUBLIC_API_URL,
  headers: { 'Content-Type': 'application/json' },
  withCredentials: true, // for httpOnly cookie auth
});

// Interceptor: attach JWT from cookie / refresh on 401
api.interceptors.response.use(
  (res) => res,
  async (error) => {
    if (error.response?.status === 401) {
      await api.post('/auth/refresh');
      return api(error.config);
    }
    return Promise.reject(error);
  }
);

export default api;

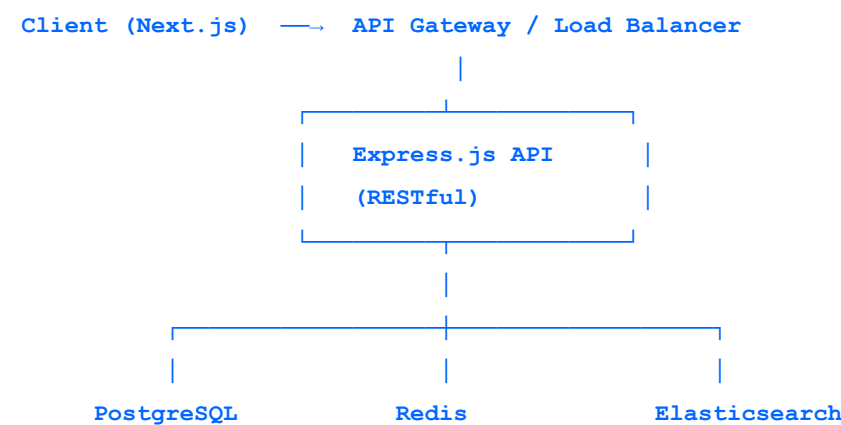
// Usage with React Query:
const { data, isLoading } = useQuery({
  queryKey: ['product', slug],
  queryFn: () => api.get(`/products/${slug}`).then(r => r.data),
  staleTime: 5 * 60 * 1000, // 5 minutes
});
```

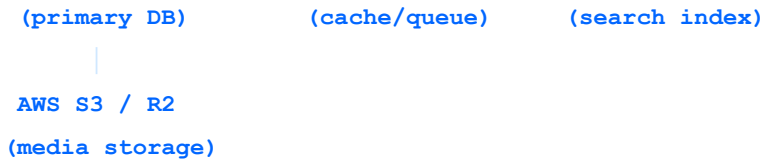
SECTION 05 **BACKEND REQUIREMENTS**

5.1 Technology Stack (I MENTIONED THE CHOICES IF YOU COMFORTABLE WITH ANY OTHER YOU CAN DO IT BY YOU OWN WHICH IS COMPACT AND BEST)

Layer	Technology
Runtime	Node.js 20 LTS
Framework	Express.js (REST APIs) — or NestJS for structured large teams
Language	TypeScript
ORM	Prisma ORM with PostgreSQL
Database	PostgreSQL 15 (primary relational DB)
Cache	Redis 7 (session, rate limiting, cart cache, product cache)
Search	Elasticsearch 8 (product search, autocomplete, faceted filters)
File Storage	Cloud Server/ VPS / Cloudflare R2 (product images, invoices, exports)
Email	Nodemailer
SMS	Twilio or AWS SNS
Payments	Razorpay
Queue	Bull + Redis (async jobs: emails, order processing, SE notifications)
Authentication	JWT (access token 15min) + refresh token (httpOnly cookie, 7d)
Process Manager	PM2 (production)
Container	Docker + Docker Compose
CI/CD	GitHub Actions → AWS ECS / Railway / Render

5.2 Architecture Overview





5.3 Authentication System

Registration

- POST /auth/register — validate fields, hash password (bcrypt, cost 12), create user, assign SE, send welcome email via queue

Login

- POST /auth/login — validate credentials, return access token (15min JWT) + set httpOnly refresh token cookie

Token Refresh

- POST /auth/refresh — validate refresh token from cookie, issue new access token

OAuth (optional v2)

- GET /auth/google — Passport.js Google OAuth2 strategy

Password Reset

- POST /auth/forgot-password → email with signed 1-hour reset link
- POST /auth/reset-password — validate token, update hashed password

5.4 Role & Permission System

Role	Access Level	Key Permissions
customer	Store only	Browse, cart, checkout, account, wishlist, reviews
support	Admin (limited)	Orders (read/update status), customers (read), reviews
manager	Admin (partial)	All above + products, blog, coupons
admin	Full access	All modules + user management, settings, roles

```
// middleware/authorize.ts
export const authorize = (...roles: Role[]) => {
  return (req: AuthRequest, res: Response, next: NextFunction) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ error: 'Forbidden' });
    }
    next();
  };
};
```



```
};
```

```
// Usage:
```

```
router.delete('/:id', authenticate, authorize('admin','manager'),  
deleteProduct);
```

5.5 Core Modules

Products Module

- CRUD operations, variant management, image uploads to S3, Elasticsearch indexing on save/update
- Soft delete (deleted_at timestamp), restore endpoint
- Bulk CSV import with validation job (Bull queue)

Orders Module

- Create order (validate stock, lock units, create payment intent)
- Status machine: pending → confirmed → processing → shipped → delivered → returned
- Webhook handler for payment gateway events (Stripe webhook, Razorpay webhook)
- Invoice generation (PDF via Puppeteer), stored in S3, linked to order

CRM / Sales Engineer Module (NOT REQUIRED)

- Auto-assign SE on user registration (round-robin by active SE workload)
- SE can be manually re-assigned by admin
- Conversation threads: customer ↔ SE messages stored in DB
- SE performance analytics: revenue generated, avg response time, CSAT score

Search Module

- Elasticsearch index: product name, brand, description, tags, category, attributes
- Autocomplete: edge ngram tokeniser for prefix matching
- Faceted filter aggregations: brand, price range, rating, availability
- Typo tolerance: fuzziness:auto for queries > 4 chars

SECTION 06 DATABASE STRUCTURE

6.1 Entity Relationship Summary

Database: PostgreSQL 15 with Prisma ORM. All tables use UUID primary keys. Soft deletes via deleted_at. Timestamps: created_at, updated_at on all tables.

6.2 Table Definitions

users

Column	Type	Notes
id	UUID PK	
email	VARCHAR(255) UNIQUE NOT NULL	
password_hash	VARCHAR(255)	NULL for OAuth users
first_name	VARCHAR(100) NOT NULL	
last_name	VARCHAR(100) NOT NULL	
phone	VARCHAR(20)	
avatar_url	TEXT	
role	ENUM('customer','sales_engineer','support','manager','admin')	Default: customer
is_verified	BOOLEAN	Default: false
is_active	BOOLEAN	Default: true
sales_engineer_id	UUID FK → users	Assigned SE
wallet_balance	DECIMAL(10,2)	Default: 0.00
created_at / updated_at	TIMESTAMPTZ	

products

Column	Type	Notes
id	UUID PK	
slug	VARCHAR(255) UNIQUE NOT NULL	URL-friendly identifier
name	VARCHAR(255) NOT NULL	
brand_id	UUID FK → brands	
category_id	UUID FK → categories	
description	TEXT	Rich HTML

Column	Type	Notes
short_description	VARCHAR(500)	
base_price	DECIMAL(10,2) NOT NULL	
sale_price	DECIMAL(10,2)	NULL if not on sale
sku	VARCHAR(100) UNIQUE NOT NULL	
stock_quantity	INTEGER NOT NULL	Total across variants
condition	ENUM('new','used','refurbished')	
status	ENUM('active','draft','archived')	
weight_kg	DECIMAL(5,2)	For shipping calc
meta_title / meta_description	VARCHAR(255) / TEXT	SEO fields
deleted_at	TIMESTAMPTZ	Soft delete
created_at / updated_at	TIMESTAMPTZ	

product_variants

Column	Type	Notes
id	UUID PK	
product_id	UUID FK → products	
sku	VARCHAR(100) UNIQUE NOT NULL	
name	VARCHAR(255)	e.g., 'Sunburst Finish'
price_modifier	DECIMAL(8,2)	Added to base_price
stock_quantity	INTEGER NOT NULL	
attributes	JSONB	e.g., {color: 'red', size: 'M'}

categories

Column	Type	Notes
id	UUID PK	
name	VARCHAR(255) NOT NULL	
slug	VARCHAR(255) UNIQUE NOT NULL	
parent_id	UUID FK → categories	NULL for root
image_url	TEXT	
is_featured	BOOLEAN	Default: false
sort_order	INTEGER	For manual ordering
meta_title / meta_description	TEXT	SEO

orders

Column	Type	Notes
id	UUID PK	
order_number	VARCHAR(20) UNIQUE	e.g., HRM-20260001
user_id	UUID FK → users	NULL for guest orders
guest_email	VARCHAR(255)	For guest checkout
status	ENUM('pending','confirmed','processing','shipped','delivered','cancelled','returned')	
subtotal	DECIMAL(10,2)	
discount_amount	DECIMAL(10,2)	
shipping_amount	DECIMAL(10,2)	
tax_amount	DECIMAL(10,2)	
total_amount	DECIMAL(10,2)	
coupon_id	UUID FK → coupons	NULL
shipping_address_id	UUID FK → addresses	
shipping_method	VARCHAR(50)	
tracking_number	VARCHAR(100)	
notes	TEXT	Customer notes at checkout
invoice_url	TEXT	S3 link
created_at / updated_at	TIMESTAMPTZ	

order_items

Column	Type	Notes
id	UUID PK	
order_id	UUID FK → orders	
product_id	UUID FK → products	
variant_id	UUID FK → product_variants	NULL if no variant
product_name	VARCHAR(255)	Snapshot at time of order
product_image	TEXT	Snapshot
unit_price	DECIMAL(10,2)	Snapshot

Column	Type	Notes
quantity	INTEGER NOT NULL	
line_total	DECIMAL(10,2)	unit_price × quantity

payments

Column	Type	Notes
id	UUID PK	
order_id	UUID FK → orders	
gateway	ENUM('razorpay','wallet','cod')	
gateway_payment_id	VARCHAR(255)	External payment reference
gateway_order_id	VARCHAR(255)	
amount	DECIMAL(10,2)	
currency	CHAR(3)	Default: USD
status	ENUM('pending','completed','failed','refunded')	
refund_amount	DECIMAL(10,2)	
metadata	JSONB	Raw gateway response
created_at / updated_at	TIMESTAMPTZ	

reviews

Column	Type	Notes
id	UUID PK	
product_id	UUID FK → products	
user_id	UUID FK → users	
order_item_id	UUID FK → order_items	Verified purchase check
rating	SMALLINT (1–5) NOT NULL	
title	VARCHAR(200)	
body	TEXT NOT NULL	
status	ENUM('pending','approved','rejected')	
helpful_votes	INTEGER	Default: 0
admin_reply	TEXT	
created_at / updated_at	TIMESTAMPTZ	

cart (persistent cart)

Column	Type	Notes
id	UUID PK	
user_id	UUID FK → users	NULL for guest (use session_id)
session_id	VARCHAR(255)	For guest cart
expires_at	TIMESTAMPTZ	30 days from last activity

cart_items

Column	Type	Notes
id	UUID PK	
cart_id	UUID FK → cart	
product_id	UUID FK → products	
variant_id	UUID FK → product_variants	
quantity	INTEGER NOT NULL	
added_at	TIMESTAMPTZ	

se_conversations & se_messages

Column	Type	Notes
se_conversations.id	UUID PK	One per customer-SE pair
se_conversations.customer_id	UUID FK → users	
se_conversations.se_id	UUID FK → users	
se_conversations.status	ENUM('open','closed')	
se_messages.id	UUID PK	
se_messages.conversation_id	UUID FK → se_conversations	
se_messages.sender_id	UUID FK → users	
se_messages.body	TEXT NOT NULL	
se_messages.is_internal	BOOLEAN	SE-only note, not visible to customer
se_messages.created_at	TIMESTAMPTZ	

blog_posts

Column	Type	Notes
id	UUID PK	
author_id	UUID FK → users	

Column	Type	Notes
title	VARCHAR(255) NOT NULL	
slug	VARCHAR(255) UNIQUE NOT NULL	
body	TEXT	Rich HTML / Markdown
excerpt	VARCHAR(500)	
cover_image_url	TEXT	
status	ENUM('draft','published','archived')	
tags	TEXT[]	PostgreSQL array
meta_title / meta_description	TEXT	SEO
published_at	TIMESTAMPTZ	
created_at / updated_at	TIMESTAMPTZ	

SECTION 07 API ENDPOINTS

Base URL: <https://api.Vibemusic.com/v1> — All protected routes require: Authorization: Bearer <access_token>

Standard error envelope: { error: string, code: string, details?: object }

Standard success envelope: { data: T, meta?: { page, limit, total } }

7.1 Authentication

Method + Path	Auth Required	Description
POST /auth/register	No	Register new customer account
POST /auth/login	No	Login, returns access token
POST /auth/logout	Yes	Clear refresh token cookie
POST /auth/refresh	Cookie	Refresh access token
POST /auth/forgot-password	No	Send reset email
POST /auth/reset-password	No (signed token)	Set new password
GET /auth/me	Yes	Get current user profile

```
// POST /auth/register
// Request Body:
{
    "first_name": "Alex",
    "last_name": "Turner",
    "email": "alex@example.com",
    "password": "SecurePass@1",
    "phone": "+911234567890"
}

// Response 201:
{
    "data": {
        "id": "uuid",
        "email": "alex@example.com",
        "access_token": "eyJhbGciOiJIc..."
    }
}
```


7.2 Products

Method + Path	Auth Required	Description
GET /products	No	List products with filters, sorting, pagination
GET /products/:slug	No	Single product detail by slug
GET /products/:id/related	No	Related products (same category)
GET /products/:id/frequently-bought-with	No	Cross-sell suggestions
POST /admin/products	admin/manager	Create new product
PUT /admin/products/:id	admin/manager	Update product
DELETE /admin/products/:id	admin	Soft delete product
POST /admin/products/bulk-import	admin/manager	CSV bulk import
PUT /admin/products/:id/images	admin/manager	Upload product images to S3

```
// GET /products?category=electric-
guitars&brand=fender&min_price=500&max_price=2000
//           &rating=4&sort=price_asc&page=1&limit=24
// Response 200:
{
  "data": [
    {
      "id": "uuid",
      "slug": "fender-player-stratocaster",
      "name": "Fender Player Stratocaster",
      "brand": { "id": "uuid", "name": "Fender" },
      "base_price": 849.99,
      "sale_price": 749.99,
      "images": [{ "url": "https://cdn.vibemusic.com/img.jpg", "alt": "...",
        "is_primary": true }],
      "average_rating": 4.7,
      "review_count": 312,
      "in_stock": true,
      "condition": "new"
    }
  ],
  "meta": { "page": 1, "limit": 24, "total": 1842 }
}
```

7.3 Cart

Method + Path	Auth Required	Description
GET /cart	Optional (guest uses session)	Get current cart with line items
POST /cart/items	Optional	Add item { productId, variantId?, quantity }
PUT /cart/items/:itemId	Optional	Update quantity { quantity }
DELETE /cart/items/:itemId	Optional	Remove item from cart
DELETE /cart	Optional	Clear entire cart
POST /cart/coupon	Yes (customer)	Apply coupon { code }
DELETE /cart/coupon	Yes (customer)	Remove applied coupon
POST /cart/merge	Yes (customer)	Merge guest cart into user cart on login

7.4 Orders

Method + Path	Auth Required	Description
POST /orders	Yes / Guest	Create order from cart
GET /orders	Yes (customer)	Customer's order history
GET /orders/:id	Yes (owner/admin)	Order detail
GET /orders/:id/track	No (with order number)	Public tracking by order number
PUT /admin/orders/:id/status	admin/support	Update order status
POST /admin/orders/:id/refund	admin/manager	Issue refund
GET /admin/orders	admin/support/manager	All orders with filters

```
// POST /orders
// Request:
{
  "cart_id": "uuid",
  "shipping_address_id": "uuid",
  "shipping_method": "standard",
  "payment_method": "stripe",
  "payment_intent_id": "pi_xxx",
  "notes": "Leave at door",
  "guest_email": null
}
// Response 201:
{
```

```
"data": {  
  "id": "uuid",  
  "order_number": "HRM-20260001",  
  "status": "confirmed",  
  "total_amount": 749.99,  
  "estimated_delivery": "2026-04-20"  
}
```

7.5 Reviews

Method + Path	Auth Required	Description
GET /products/:id/reviews	No	Paginated product reviews
POST /products/:id/reviews	Yes (customer)	Submit review { rating, title, body, orderItemId }
POST /reviews/:id/helpful	Yes (customer)	Mark review as helpful
GET /admin/reviews	admin/manager/support	All reviews with status filter
PUT /admin/reviews/:id/status	admin/manager	Approve or reject review { status }
POST /admin/reviews/:id/reply	admin/support	Add admin reply { reply }

7.6 Search

Method + Path	Auth Required	Description
GET /search?q=:query&limit=10	No	Autocomplete suggestions
GET /search/results?q=:q&filters=...&sort=:s&page=:p	No	Full search results with facets
GET /search/trending	No	Trending search terms

7.7 Sales Engineer (CRM)

Method + Path	Auth Required	Description
GET /me/sales-engineer	Yes (customer)	Get my assigned SE details
GET /se/customers	Yes (SE)	SE's assigned customer list
GET /se/conversations	Yes (SE)	All SE message threads
GET /conversations/:id/messages	Yes (SE/customer)	Messages in a thread
POST /conversations/:id/messages	Yes (SE/customer)	Send message { body, is_internal? }
POST /admin/se/assign	Yes (admin)	Assign SE { customer_id, se_id }

Method + Path	Auth Required	Description
GET /admin/se/performance	Yes (admin)	SE performance metrics

7.8 Payments

Method + Path	Auth Required	Description
POST /payments/razorpay/order	Yes	Create Razorpay order
POST /payments/webhook/razorpay	No (Razorpay signature)	Razorpay event handler

SECTION 08

ADMIN PANEL REQUIREMENTS

8.1 Technology Stack (Admin Panel)

The admin panel is a separate Next.js application (or a route group /admin within the main app) protected by role-based middleware.

Layer	Choice
Framework	Next.js 14 App Router (shared codebase with storefront, /admin route group)
UI Library	shadcn/ui + Tailwind — clean, accessible admin components
Data Tables	TanStack Table v8 — sortable, filterable, paginated
Charts	Recharts — revenue, orders, traffic dashboards
Rich Text Editor	TipTap — for blog/product description editing
File Upload	react-dropzone + S3 presigned URL upload
State	React Query (server state) + Zustand (UI state)

8.2 Dashboard Analytics (Admin Home)

- Revenue Cards: Today / This Week / This Month — with % change vs prior period
- Order Cards: New Orders / Pending / Processing / Shipped
- Customer Stats: New Registrations Today / Total Active Customers
- Revenue Chart: Line chart with 30-day revenue and comparison period overlay
- Top Products Table: Product name, units sold, revenue this month
- Low Stock Alerts: Products with stock < 10 units — sorted by urgency
- Recent Orders Feed: Live updating list of last 20 orders with status
- Search Term Cloud: Top 20 search terms this week

8.3 Product Management

Product List Page

- Searchable, filterable data table: name, SKU, category, price, stock, status
- Bulk actions: activate, deactivate, delete, update category
- Export to CSV button
- Import via CSV with column mapper UI and validation preview

Product Create / Edit Form

- Tab 1 — Basic Info: name, slug (auto-generated), brand, categories, condition, status
- Tab 2 — Pricing & Inventory: base price, sale price, cost price, SKU, stock quantity, low-stock threshold
- Tab 3 — Images: drag-drop multi-upload, reorder (drag), set primary, alt text per image
- Tab 4 — Variants: add colour/size/finish options → auto-generate variant SKUs
- Tab 5 — Description: TipTap rich editor with formatting toolbar

- Tab 6 — Specs: dynamic key-value table for technical specifications
- Tab 7 — SEO: meta title, meta description (char count), OG image

8.4 Order Management

- Orders table: order number, customer, date, total, status badge, assigned SE
- Filter: by status, date range, shipping method, payment gateway
- Order detail page: items, customer info, shipping address, payment info, timeline
- Status update dropdown with confirmation modal
- Refund panel: full or partial refund, reason selection, gateway refund trigger
- Add tracking number, courier name → auto-sends tracking email to customer
- Print invoice / packing slip (PDF generated via API)

8.5 Customer Management

- Customer table: name, email, total orders, total spent, SE assigned, registration date
- Customer detail: full profile, order history, communication history with SE, notes
- Assign / re-assign Sales Engineer via dropdown
- Flag account / ban account (with reason)
- Impersonate customer (for support purposes) — admin only

8.6 Sales Engineer Assignment Panel (NOT REQUIRED)

- SE Roster Table: SE name, active customers count, open conversations, avg response time, monthly revenue
- Unassigned customers queue: auto-assign or manual assign via dropdown
- Reassignment interface: drag customer card from one SE column to another (Kanban-style)
- SE performance report: downloadable monthly PDF report per SE
- SE Notes viewer: admin can read all internal SE notes for compliance

8.7 Content / Blog Management

- Blog post list table: title, author, status, published date, views
- Create/edit post: TipTap editor, cover image upload, tags, category, SEO fields
- Publish / Draft / Schedule (set future publish date)
- Homepage banner manager: upload banner image, set CTA text, link URL, sort order, date range
- Featured categories selector: toggle which categories appear in homepage grid

SECTION 09 CHECKOUT & PAYMENT FLOW

9.1 Step-by-Step Checkout Flow

CART PAGE

- User reviews items, adjusts quantities, applies coupon
- Clicks 'Proceed to Checkout'

STEP 1 — CONTACT & SHIPPING ADDRESS

- Guest: enter email, name, phone
- Logged-in: pre-fill from saved profile, select saved address or add new
- Google Places autocomplete on address field
- Click 'Continue to Shipping'

STEP 2 — SHIPPING METHOD

- API call: GET /shipping/rates?cartId=:id&addressId=:id
- Display options (Standard Free / Express \$9.99 / Overnight \$24.99) with ETA dates
- User selects method → Order Summary updates in sidebar
- Click 'Continue to Payment'

STEP 3 — PAYMENT

- API call: POST /payments/stripe/intent → returns client_secret
- Stripe Elements card form renders (PCI-compliant embedded iframe)
- Additional options: Razorpay (UPI/NetBanking), Wallet Balance, Affirm BNPL
- User enters payment details
- Click 'Continue to Review'

STEP 4 — ORDER REVIEW

- Full summary: items, shipping address, shipping method, payment method, total
- 'Edit' links back to relevant steps
- 'Place Order' button
- On click: Disable button, show overlay loader
- Frontend: stripe.confirmCardPayment(clientSecret, { payment_method: {...} })
- On Stripe success: POST /orders with payment_intent_id

STEP 5 — CONFIRMATION

- Order number displayed with animation
- Item list + estimated delivery date

- 'Track Your Order' CTA → /account/orders/:id
- Transactional email sent (queued via Bull → SES/SendGrid)
- Cart cleared

9.2 Payment Gateway Integration

Razorpay

- Create Razorpay order server-side, return order_id to frontend
- Frontend: open Razorpay checkout modal with order_id and key_id (public)
- On payment success: POST /payments/webhook/razorpay with razorpay_signature for HMAC verification

9.3 Order Tracking (NOT REQUIRED)

Tracking is available both from the account dashboard and via a public URL with order number + email verification.

Status	Customer-Facing Label	Trigger
pending	Order Received	Order created, payment pending
confirmed	Order Confirmed	Payment webhook success received
processing	Being Prepared	Admin manually updates or WMS webhook
shipped	Shipped	Admin adds tracking number
out_for_delivery	Out for Delivery	Carrier webhook (if integrated)
delivered	Delivered	Carrier webhook or admin update
returned	Return Initiated	Customer submits return request

SECTION 10 UX & DESIGN GUIDELINES

10.1 Design Principles

- Music-first aesthetic: Bold, passionate, professional — not sterile corporate
- Trust signals everywhere: ratings, reviews, verified badges, secure payment icons
- Friction-less journey: Minimize clicks to purchase. Guest checkout, one-click re-order
- Clarity over cleverness: Every UI element should have an obvious purpose
- Mobile-first: Design for phone first, enhance for tablet and desktop

10.2 Colour System

Token	Value & Usage
--color-brand-navy	#1A3A5C — Primary brand, headers, trust elements
--color-brand-orange	#E8461A — CTAs (Add to Cart, Buy Now), accent highlights
--color-brand-blue	#2E75B6 — Links, interactive elements, info states
--color-surface-1	#FFFFFF — Card backgrounds, main content areas
--color-surface-2	#F4F6F8 — Page background, alternating table rows
--color-surface-3	#EBF2FA — Subtle highlights, feature boxes
--color-text-primary	#1A1A2E — Main body text
--color-text-secondary	#5A6472 — Labels, captions, secondary info
--color-success	#16A34A — In-stock, success states, confirmations
--color-warning	#D97706 — Low stock, warnings
--color-error	#DC2626 — Form errors, out-of-stock, critical alerts
--color-border	#CBD5E1 — Dividers, card borders

10.3 Typography

Element	Spec
H1 — Page Titles	Inter / 36px / Bold 700 / #1A3A5C
H2 — Section Titles	Inter / 28px / SemiBold 600 / #1A3A5C
H3 — Card Titles, Subsections	Inter / 22px / SemiBold 600 / #1A3A5C
Body — Default	Inter / 16px / Regular 400 / #1A1A2E / line-height: 1.6
Small / Caption	Inter / 13px / Regular 400 / #5A6472
Product Price	Inter / 24px / Bold 700 / #1A1A2E
Sale Price	Inter / 24px / Bold 700 / #E8461A

Element	Spec
Label / Badge	Inter / 11px / SemiBold 600 / Uppercase / Letter-spacing: 0.05em
Code / SKU	JetBrains Mono / 13px / #5A6472

10.4 Component Design Standards

Product Card

- White background, subtle border (1px #CBD5E1), 8px border-radius, shadow on hover
- Image: 1:1 aspect ratio, object-cover, background #F4F6F8
- 'Quick Add' button appears on hover (desktop) — always visible on mobile
- Wishlist heart icon top-right, filled on hover/saved
- Rating: filled/half/empty stars + count in parentheses

Buttons

- Primary (Add to Cart): bg-brand-orange, white text, 8px radius, 48px height min, hover: darken 8%
- Secondary (View Details): border-brand-navy, navy text, transparent bg, hover: bg-brand-navy + white text
- Ghost: no border, text colour link, hover: underline
- Disabled: opacity-50, cursor-not-allowed
- Loading state: replace label with spinner, disable pointer events

Forms

- Label always above input (not placeholder-only — accessibility)
- Error message: red text below field + red border on input
- Input height: 44px minimum (touch-friendly)

10.5 Navigation UX

- Mega Menu: opens on hover (desktop) with 300ms delay to prevent accidental trigger; opens on tap (mobile)
- Mobile navigation: full-screen drawer from left, accordion category tree
- Sticky header: on scroll > 80px, header shrinks (reduces height from 80px to 60px)
- Active category highlighted in breadcrumb and sidebar nav
- Back to top button appears after 400px scroll on mobile

SECTION 11 PERFORMANCE & SECURITY

11.1 Performance Requirements

Metric	Target	Implementation
LCP (Largest Contentful Paint)	< 2.0s	Next/Image, hero image preload, CDN delivery
FID / INP	< 100ms	Code splitting, avoid main thread blocking
CLS (Cumulative Layout Shift)	< 0.1	Reserve image dimensions, avoid late-loading ads
TTFB	< 400ms	Redis cache for product/category pages, CDN edge caching
Bundle size (JS)	< 150KB (initial)	Dynamic imports, tree shaking, next/bundle-analyzer
Product images	< 100KB each	WebP format, auto-compress on S3 upload, responsive srcsets

Caching Strategy

- Static pages (Homepage, Category): ISR (Incremental Static Regeneration) revalidate: 300s
- Product pages: ISR revalidate: 60s, Redis cache for product detail: TTL 5min
- Search results: No static cache — always dynamic
- Cart / User data: No caching — always fresh from DB
- Images: CloudFront CDN with 1-year cache-control (URL versioning on upload)
- API responses: ETag + If-None-Match headers for conditional requests

11.2 Security Requirements

Authentication Security

- Access tokens: JWT, RS256 algorithm, 15-minute expiry
- Refresh tokens: httpOnly, SameSite=Strict, Secure cookies, 7-day expiry, rotated on each use
- Failed login attempts: rate limit 5 per 10 minutes per IP + per email (Redis)
- Password reset links: signed JWT, 1-hour expiry, single-use (invalidated after use)
- 2FA: TOTP-based (Google Authenticator) — optional v2 feature

API Security

- All API inputs: validated with Zod schemas, malformed requests rejected with 400
- SQL Injection: Prisma parameterised queries — no raw SQL without explicit sanitisation
- XSS: All HTML output sanitised with DOMPurify (server-side: sanitize-html)
- CORS: Allow only trusted origins (production domain + admin domain)
- Rate limiting: 100 req/min (public), 300 req/min (authenticated) — express-rate-limit + Redis
- Helmet.js: security headers (CSP, HSTS, X-Frame-Options, etc.)
- Admin routes: require role check + IP allowlist (optional — configurable per deployment)

Payment Security

- PCI-DSS: Card data never touches Vibe Music servers — Stripe Elements in iframe
- Webhook signature verification: Stripe-Signature header, Razorpay HMAC-SHA256 verification
- Payment amounts: always calculated server-side (never trust frontend total)
- Order locking: Stock reserved atomically on order creation, released on payment failure/timeout

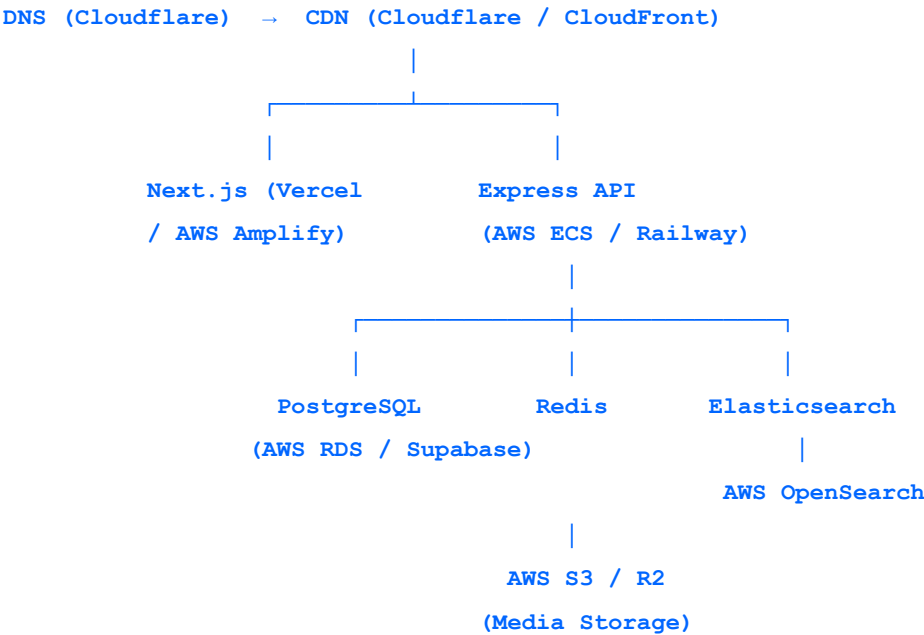
Data Security

- HTTPS everywhere — TLS 1.3 minimum
- Database: encrypted at rest (AWS RDS encryption or Supabase)
- S3 buckets: private by default, public only via CloudFront signed URLs
- Secrets management: AWS Secrets Manager / HashiCorp Vault — no secrets in .env committed to git
- GDPR/PDPA compliance: data deletion endpoint, consent logs, cookie banner

SECTION 12 FINAL DELIVERY & INFRASTRUCTURE

12.1 Infrastructure Architecture

PRODUCTION INFRASTRUCTURE:



12.2 Environment Configuration

Variable	Description	Example / Notes
DATABASE_URL	PostgreSQL connection string	postgresql://user:pass@host:5432/Vibemusic
REDIS_URL	Redis connection	redis://default:pass@host:6379
ELASTICSEARCH_URL	Elasticsearch endpoint	https://host:9200
JWT_PRIVATE_KEY	RS256 private key for signing tokens	PEM encoded, from Secrets Manager
JWT_PUBLIC_KEY	RS256 public key for verification	PEM encoded
RAZORPAY_KEY_ID	Razorpay public key	rzp_live_xxx

Variable	Description	Example / Notes
RAZORPAY_KEY_SECRET	Razorpay secret key	From Razorpay dashboard
CLOUD/VPS_BUCKET	S3 bucket name	Vibe Music-media-prod
CLOUD/VPS_REGION	CLOUD/VPS region	ap-south-1
SENDGRID_API_KEY / SES config	Email service credentials	
TWILIO_ACCOUNT_SID / AUTH_TOKEN	SMS service	
GOOGLE_PLACES_API_KEY	Address autocomplete	Frontend-safe key (restricted by domain)
NEXT_PUBLIC_API_URL	Backend API base URL (frontend)	https://api.Vibe Music.com/v1
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY	Stripe public key (frontend)	pk_live_xxx

12.3 Development Workflow

Git Branching Strategy (GitFlow)

- main — production-ready code only, protected branch
- develop — integration branch for feature PRs
- feature/[ticket-id]-description — individual feature branches
- hotfix/[ticket-id]-description — production bug fixes
- release/v[version] — release preparation branch

CI/CD Pipeline (GitHub Actions)

6. On PR to develop: run lint, type-check, unit tests, E2E tests (Playwright headless)
7. On merge to develop: deploy to staging environment automatically
8. On merge to main: deploy to production with approval gate
9. Docker image built, tagged with git SHA, pushed to ECR
10. ECS service updated with rolling deployment (zero downtime)

12.4 Project Kickoff Checklist for Developers

Backend Team — Day 1 Setup

- Clone repo, install dependencies, copy .env.example to .env
- docker compose up -d (PostgreSQL, Redis, Elasticsearch)
- npx prisma migrate dev (run initial migrations)
- npx prisma db seed (seed categories, admin user, test products)
- npm run dev — confirm API running on :4000

Frontend Team — Day 1 Setup

- Clone repo, npm install, copy .env.local.example to .env.local
- Set NEXT_PUBLIC_API_URL=http://localhost:4000/v1
- npm run dev — confirm app running on :3000
- Visit /admin with seeded admin@Vibe Music.com / Admin@12345 credentials

Admin Panel Team

- Admin panel is at /admin route — same Next.js app, separate layout group
- Use seeded admin account. Test: product creation, order status update, SE assignment

12.5 Milestone Plan

Milestone	Deliverables	Estimated Duration
M1 — Foundation	DB schema, auth module, product CRUD API, basic category page	
M2 — Core Store	Homepage, category page, product page, search (Elasticsearch)	
M3 — Commerce	Cart, checkout (Razorpay), order creation, confirmation email	
M4 — Admin Panel	Full admin dashboard, product mgmt, order mgmt, customer mgmt	
M6 — Polish	Reviews, wishlist, blog, performance optimisation, accessibility audit	
M7 — Testing & Launch	E2E test suite, security audit, staging UAT, production deploy	

END OF DOCUMENT

Vibe Music eCommerce Platform — Website Requirement Document v1.0